1) What are Semantic Elements & Non-Semantic Elements?

Semantic Elements

These elements describe their meaning to both developers and browsers.

Examples:

<header>

<nav>

<main>

<section>

<article>

<aside>

<footer>

<form>

<button>

Advantages

- Better SEO
- Better Accessibility
- Easier maintenance
- Screen readers understand page structure

Example

```
<header>
  <nav></nav>
</header>

<main>
  <section>
    <article></article>
  </section>
</main>

<footer></footer>
```

Non-Semantic Elements

They don't describe their content.

Examples

Need CSS/classes to explain purpose.

---

2) Suppose there are multiple buttons on a page, and you want to style only the last button. How would you do it?

Using CSS pseudo selector.

```
button:last-child{
  background:red;
}
```

or

```
button:last-of-type{
  background:red;
}
```

If inside a container

```
.container button:last-child{
  color:white;
}
```

---

3) What is Accessibility?

Accessibility (A11Y) means making websites usable for everyone, including people with disabilities.

Examples

- Keyboard navigation
- Screen reader support
- Proper color contrast
- Alt text
- Labels for inputs
- Semantic HTML
- ARIA attributes

Example

❌

```
<div onclick="save()">Save</div>
```

✅

```
<button>Save</button>
```

Angular tools

- Angular CDK Accessibility
- Lighthouse
- Axe DevTools

---

4) What are the different ways to center elements in CSS?

Flexbox

```
display:flex;
justify-content:center;
align-items:center;
```

Grid

```
display:grid;
place-items:center;
```

Margin Auto

```
width:300px;
margin:auto;
```

Position

```
position:absolute;
top:50%;
left:50%;
transform:translate(-50%,-50%);
```

Text

```
text-align:center;
```

---

5) How do you write Responsive CSS?

Methods

- Flexbox
- CSS Grid
- Media Queries
- Relative Units
- Responsive Images

Example

```
.container{
 display:flex;
 flex-wrap:wrap;
}

@media(max-width:768px){
  .container{
    flex-direction:column;
  }
```

}

---

6) What are the common CSS breakpoints?

Mobile
0–576px

Tablet
577–768px

Laptop
769–992px

Desktop
993–1200px

Large Desktop
1200px+
Example

```
@media(max-width:576px){}

@media(max-width:768px){}

@media(max-width:992px){}

@media(min-width:1200px){}
```

---

7) Which CSS units do you prefer for responsive layouts?

Unit	Use
rem	Fonts, spacing
em	Component-relative sizing
%	Width
vw	Full-width layouts
vh	Full-height layouts
fr	CSS Grid
clamp()	Responsive typography

Example

font-size:clamp(16px,2vw,24px);

---

8) Can you explain the JavaScript Event Loop?
JavaScript is **single-threaded**.
The Event Loop continuously checks:
Call Stack empty?

↓

Yes

↓

Move Microtasks

↓

Render

↓

Move Macrotasks

---

9) How do the Call Stack, Web APIs, Callback Queue, and Microtask Queue work together?
JS Code

↓

Call Stack

↓

Web APIs
(setTimeout, fetch)

↓

Callback Queue
(setTimeout)

Microtask Queue
(Promise)

↓

Event Loop

↓

Call Stack
Priority
Microtasks

↓

Macrotasks

---

10) Which executes first: Promises or setTimeout() callbacks?

Example

```
console.log(1);

setTimeout(()=>{
  console.log(2);
},0);

Promise.resolve().then(()=>{
  console.log(3);
});

console.log(4);
```

Output

1
4
3
2

Because
Promise → Microtask
setTimeout → Macrotask
Microtasks execute first.

---

11) What is the difference between call(), apply(), and bind()?
call()
Immediately invokes.
person.call(obj,"John");

---

apply()
Immediately invokes.
Arguments as array.
person.apply(obj,["John"]);

---

bind()
Returns a new function.
const fn = person.bind(obj);

fn();

---

12) What is the difference between a Deep Copy and a Shallow Copy?
Shallow Copy
Copies first level only.
const copy = {...obj};
Nested objects still reference same memory.

Deep Copy
Everything copied.
structuredClone(obj)
or
_.cloneDeep(obj)

---

13) Where have you used Debouncing in Angular applications?
Common uses

- Search box
- API calls
- Auto-save

- Filtering
- Resize events

Example

```
search.valueChanges.pipe(
 debounceTime(300),
 distinctUntilChanged(),
 switchMap(...)
)
```

---

14) What are the advantages of Lazy Loading, and how do you implement it in Angular?

Advantages

- Smaller bundle
- Faster startup
- Better SEO
- Better Core Web Vitals

Routes

```
{
 path:'admin',
 loadChildren:()=>
 import('./admin/admin.routes')
}
```

---

15) How is Lazy Loading implemented with Standalone Components?

```
{
 path:'dashboard',
 loadComponent:()=>
 import('./dashboard.component')
 .then(c=>c.DashboardComponent)
}
```

No NgModule needed.

---

16) Can you explain Angular Change Detection?

Angular checks whether model values changed.

Strategies

Default

Checks entire component tree.

OnPush

Checks only when

- Input changes
- Event occurs
- Signal updates
- Observable emits (async pipe)
- markForCheck() is called

OnPush improves performance.

---

17) What are Standalone Components? What are their advantages?

Standalone components remove the need for NgModules.

```
@Component({
 standalone:true,
 imports:[CommonModule]
})
```

Advantages

- Less boilerplate
- Better lazy loading
- Faster builds
- Simpler architecture
- Easier testing

---

18) Which Lifecycle Hook would you use to clean up subscriptions?

ngOnDestroy()

Angular 16+

Better

takeUntilDestroyed()

No manual unsubscribe.

---

19) Let's say you have a DatePicker library you are using in your project. Which Lifecycle Hook is best to initialize or manipulate it?

Use

ngAfterViewInit()

Because DOM is ready.

```
ngAfterViewInit(){
 new DatePicker(...)
}
```

---

20) How can we optimize an Angular application's performance?

- Lazy Loading

- Standalone Components
- Signals
- OnPush
- trackBy
- Virtual Scrolling
- Route Preloading
- Pure Pipes
- Memoization
- Image Lazy Loading
- Bundle optimization
- SSR/Hydration (if applicable)
- Avoid unnecessary subscriptions

---

21) What coding patterns do you follow for better Angular performance?

- OnPush Change Detection
- Signals for local state
- Async Pipe
- trackBy
- Smart/Dumb components
- Lazy loading
- Standalone components
- Avoid logic in templates
- Pure pipes
- Use inject() instead of constructor where appropriate
- Avoid nested subscriptions (switchMap, mergeMap, etc.)

---

22) What techniques have you used in your project to improve performance?

Example answer:

In my recent Angular 19 projects, I used standalone components with route-level lazy loading to reduce the initial bundle size. I adopted OnPush change detection and Signals for local state management, minimizing unnecessary UI updates. Lists used trackBy and virtual scrolling for large datasets. API calls were optimized with debouncing, caching, and request cancellation using switchMap. Images were lazy-loaded, heavy third-party libraries were imported only where needed, and production builds were optimized through tree shaking and code splitting.

---

23) What are all the possible ways of Component Communication in Angular?

- @Input()

- @Output() + EventEmitter
- Shared Service with RxJS
- Shared Service with Signals
- ViewChild
- ContentChild
- Template Reference Variables
- Router State / Route Parameters
- Global State (NgRx, Signal Store)
- Dependency Injection
- Input Signals (Angular 17+)

---

24) Why were Signals introduced?

Signals were introduced to provide **fine-grained reactive state management** with better performance and simpler code than relying on broad change detection cycles.

Benefits:

- Automatic dependency tracking
- More predictable updates
- Better performance
- Simpler local state management
- Integrates well with OnPush

---

25) What are the advantages of Signals over RxJS or traditional Change Detection?

Signals	**RxJS**
Synchronous by default	Great for async streams
Fine-grained UI updates	Can trigger broader updates depending on usage
Less boilerplate	Operators and subscriptions add complexity
No manual unsubscribe for signals	Subscriptions often require cleanup
Ideal for component state	Ideal for events, HTTP, WebSockets, streams

A common approach is to **use Signals for local UI state** and **RxJS for asynchronous data flows**.

---

26) How does Deployment happen in your project on different environments?

A typical enterprise deployment pipeline:

1. Developers merge code into the main branch.
2. CI pipeline (e.g., GitHub Actions, Jenkins, Azure DevOps) runs:
    - Linting
    - Unit tests

    - Build
    - Security scans
3. Angular is built with environment-specific configuration:
4. ng build --configuration=dev
5. ng build --configuration=qa
6. ng build --configuration=uat

ng build --configuration=production

7. Artifacts are deployed to the target environment (e.g., Nginx, cloud storage, Kubernetes).
8. Environment-specific API URLs and settings are supplied via configuration files or runtime configuration.
9. Smoke tests and monitoring verify the deployment before wider rollout.

Typical environments:

- Development
- QA/Test
- UAT
- Production

---

27) Do you estimate your story points?

Yes. In Agile teams, story points estimate the **relative effort, complexity, risk, and uncertainty** of a user story—not just development time.

A common approach is **Planning Poker** with a Fibonacci sequence:

1, 2, 3, 5, 8, 13, 21

When estimating, I consider:

- Functional complexity
- Technical complexity
- Dependencies
- Unknowns or risks
- Testing effort
- Integration work

For example, adding a simple form validation might be **2 points**, while implementing a new OAuth authentication flow with backend integration could be **8 or 13 points**, depending on the scope.